

Integrantes:

Anna Carolina Rezende Garrido - RA: 10443894

Gabriel Franco da Rosa Messoria - RA: 10438405

Jonas Fernando Nascimento de Melo - RA: 10443398

Verificador de Similaridade de Texto

Projeto 2 - Estruturas de Dados 2

Introdução

Este projeto implementa um sistema para análise de similaridade entre documentos de texto, com três modos de operação: lista, topK e busca. Integra pré-processamento textual, com duas funções de hashing, vetorização de frequências, cálculo da similaridade do cosseno e armazena os resultados em uma árvore AVL.

Metodologia

Neste projeto, a estrutura do sistema consiste em três elementos fundamentais: uma tabela hash (Hash Table), uma árvore AVL (AVL Tree) e um método de similaridade de cosseno (CSS).

A tabela hash (Hash Table) possui uma dimensão dinâmica, determinada pelo tamanho do texto, e utiliza um array de objetos NoHash para armazenar os termos e sua cardinalidade. Duas funções hash são utilizadas: a primeira é baseada no algoritmo hash clássico para strings, e a segunda é derivada do método hashCode nativo de uma determinada palavra, atuando como uma etapa incremental no processo de hash duplo. Essa abordagem reduz a concentração de chaves em posições adjacentes e promove uma distribuição mais uniforme dos elementos dentro da estrutura. O gerenciamento de colisões é realizado exclusivamente por meio de hashes duplos, garantindo a travessia uniforme de posições alternativas durante a adição e a recuperação, visto que ambas as operações seguem a mesma ordem de verificação.

A Árvore AVL é responsável por guardar os resultados das comparações de similaridade. Cada nó possui um valor de similaridade, uma coleção de pares de documentos que compartilham esse valor, referências para os ramos esquerdo e direito, além de sua altura na estrutura. Após o processamento de todos os arquivos, o sistema executa as comparações entre cada par possível. O grau de semelhança de cada dupla é obtido através da Similaridade do Cosseno, e esse valor é então inserido na Árvore AVL, que preserva seu equilíbrio por meio de operações de rotação. A cada nova inserção, o algoritmo recalcula as alturas dos nós e, ao identificar qualquer desequilíbrio, executa as rotações apropriadas (Simples à Direita, Simples à Esquerda ou Duplas) para manter a propriedade de balanceamento. Por fim, os dados são recuperados da AVL de forma ordenada ou seletiva, conforme o modo de operação selecionado (lista, topK ou busca). O processo de inserção na AVL obedece ao padrão estabelecido: o novo valor de similaridade é confrontado com o do nó atual, direcionando a navegação para o ramo esquerdo ou direito. Caso os valores sejam idênticos, o par é acrescentado à lista existente no nó. Concluída a inserção, a altura do nó é atualizada e o fator de

balanceamento é avaliado. Quando este ultrapassa os limites aceitáveis, a estrutura executa a rotação necessária para restaurar o equilíbrio.

Para inserir na tabela hash segue uma lógica separada. Inicialmente, os valores h_1 e h_2 são calculados. A posição inicial corresponde a h_1 e, se essa posição já estiver ocupada por outro elemento, o algoritmo incrementa o contador de tentativas e estabelece um novo índice por meio de hash duplo. Assim que uma posição correspondente é encontrada, o contador de elementos é incrementado se o elemento já existir, ou um novo registro é criado.

Descrição do Algoritmo

O sistema lê e pré-processa cada documento, removendo pontuação, stopwords e padronizando o caso das letras. Em seguida, ele insere cada palavra significativa na Tabela Hash usando double hashing para acumular suas frequências.

Análise Experimental

A fim de validar a eficiência das estruturas de dados e algoritmos implementados, realizamos uma análise de desempenho com foco em três áreas principais: o tempo de execução total, o custo de balanceamento da Árvore AVL e a eficácia da Tabela Hash.

1. Tempo de Execução

Foi cronometrando três etapas principais do processo, utilizando 5 documentos de texto como entrada. Os tempos exatos registrados durante a execução, no teste o modo lista foi o utilizado, além disso o limiar era de 0.5. Os resultados foram:

Etapa	Descrição	Tempo (ms)
1. Leitura	Carregar os 5 documentos, processá-los e criar as Tabelas Hash correspondentes.	6 ms
2. Comparação/AVL	Calcular 10 similaridades e inserir os resultados na AVL.	2 ms
3. Query (Consulta)	Executar a busca na AVL pelos resultados.	1 ms
	Total:	9 ms

A Etapa 1 (Leitura e Hashing) foi a mais lenta devido às operações de I/O e ao processamento de texto. A Etapa 2 (Comparação e Inserção AVL), apesar de computacionalmente pesada, foi mais rápida que a leitura. A Etapa 3 (Consulta) foi quase instantânea, mostrando a eficiência da Árvore AVL para buscas.

2. Análise da Árvore AVL

Implementamos uma Árvore AVL para armazenar resultados de similaridade. Comparamos 45 pares únicos de 10 documentos ($N \times (N - 1) / 2$), inserindo cada par na árvore. Durante as 45 inserções, realizamos 28 rotações para manter o balanceamento. Em média, a árvore se reequilibrou a cada 1,6 inserções, distribuindo o custo de manutenção. A classe AVL garantiu que a árvore nunca degenerasse, mantendo consultas eficientes.

3. Análise da Tabela Hash (Colisões)

Arquivo	Palavras (Itens)	Ocupação da Tabela	Colisões
arquivo1.txt	30	6%	0
arquivo2.txt	29	5.8%	0
arquivo3.txt	24	4.8%	2
arquivo4.txt	26	5.2%	0
arquivo5.txt	31	6.2%	4
arquivo6.txt	26	5.2%	4
arquivo7.txt	32	6.4%	0
arquivo8.txt	28	5.6%	6
arquivo9.txt	27	5.4%	2
arquivo10.txt	32	6.4%	0
Total/Média	285 (Total)	5.7% (Média)	18 (Total)

Processamos cerca de 50% dos arquivos, incluindo os dois maiores, sem colisões, demonstrando a capacidade do algoritmo para lidar com alta carga. Algumas colisões ocorreram, mas com impacto prático mínimo. O fator de carga médio foi de 5.7%, indicando que mais de 94% da tabela estava livre. Com baixa densidade, a média de 1.8 colisões por arquivo não comprometeu a performance. A ocorrência de colisões em arquivos específicos, como o arquivo8.txt (6 col./isões), enquanto outros similares não tiveram nenhuma, sugere que esses agrupamentos são características do vocabulário, não do algoritmo. A função de Double Hashing se mostrou eficiente e adequada para o conjunto de dados.

Resultados e Discussão

1. Análise dos Pares de Maior Similaridade

Para verificar a eficácia do algoritmo na identificação de tópicos, executamos o programa no modo "topK 5". Abaixo estão os 3 pares de documentos com a maior similaridade de cosseno em nosso conjunto de teste:

```
arquivo1.txt <-> arquivo2.txt = 0,6384
arquivo5.txt <-> arquivo4.txt = 0,3334
arquivo2.txt <-> arquivo5.txt = 0,3048
```

O programa analisou os pares corretamente. A filtragem de stopwords e o cálculo do cosseno identificaram tópicos comuns. A pontuação de 0,6385 evidencia uma similaridade relevante entre os arquivos.

2. Discussão sobre a Árvore AVL

A Árvore AVL foi crucial para o projeto. A consulta (Etapa 3) foi quase instantânea, mostrando a eficiência da busca $O(\log n)$. Substituí-la teria impactos:

- * **Max-Heap:** Excelente para operações “topK” (extração dos K maiores elementos em tempo $O(k \log n)$). Contudo, sua eficiência diminui drasticamente em operações “lista” (busca de elementos acima de um determinado limiar), que exigem uma varredura completa do heap, resultando em tempo de execução $O(n)$.
- * **Vetor Ordenado: Oferece consultas** “topK” triviais, com tempo de execução $O(k)$, e operações “lista” bastante eficientes, com tempo de execução $O(\log n + m)$. No entanto, a fase de construção inicial do vetor ordenado pode ser significativamente custosa. Cada inserção tem o potencial de exigir operações até mesmo $O(n)$ para manter a ordem, totalizando um tempo de execução de construção da ordem $O(n^2)$, o que é muito pior que os $O(n \log n)$ proporcionados por uma árvore AVL.

A AVL se mostrou a estrutura mais equilibrada, com excelente performance de inserção e alta eficiência para ambas as consultas.

3. Conclusões

O projeto entregou um sistema funcional integrando pré-processamento de texto, vetorização em Tabela Hash e ordenação em Árvore AVL. O principal desafio técnico foi a implementação da Árvore AVL, garantindo rotações duplas corretas e contagem de altura precisa após inserções recursivas. A Análise Experimental mostrou que, apesar da eficiência da Tabela Hash (7 colisões), o custo de manter a AVL balanceada foi real (5 rotações para 10 inserções). A AVL pagou um custo quase irrelevante.

Referências

<https://docs.oracle.com/javase/8/docs/api/java/io/BufferedReader.html>

https://www.w3schools.com/dsa/dsa_data_avltrees.php

https://www.w3schools.com/dsa/dsa_theory_hashtables.php

Slide: “tabela de dispersão 1, 2 e 3”

Slide: “árvore AVL”